Algoritmos de Divisão e Conquista

Prof. Ricardo P. Mesquita

Introdução

A estratégia de divisão e conquista para a resolução de problemas – por meio de *decomposições* e *recombinações* – fornece um método poderoso e versátil para o projeto de algoritmos.

Alguns exemplos, ilustrando a grande aplicabilidade de divisão e conquista, podem ser dados por algoritmos:

- de busca em tabelas, como buscas sequencial e binária;
- de classificação, como classificação por seleção (*selection sort*), por intercalação (*mergesort*) e por particionamento (*quicksort*);
- para multiplicação (de matrizes e de números binários, por exemplo);
- para seleção (para determinar máximo e mínimo, o k-ésimo máximo etc.).

Introdução

Assim:

- A estratégia de divisão e conquista opera decompondo um problema em subproblemas independentes, resolvendo-os e combinando as soluções obtidas em uma solução para o problema original.
- *Isso dá um processo recursivo de decomposições e recombinações.*

Conceito

Exemplo: suponha que desejamos achar o somatório dos elementos de uma lista de números. Podemos aplicar a ideia de dividir para conquistar:

- Para calcular o somatório dos números de uma lista L, podemos proceder da seguinte maneira.
  - Se a lista L é curta (digamos, comprimento no máximo 1), já sabemos calcular seu somatório.
  - Caso contrário, podemos partir a lista L em sublistas L_1 e L_2 e aplicar recursivamente o mesmo processo até que tenhamos só sublistas curtas, as quais já sabemos tratar;
  - então bastará somar as saídas correspondentes a essas sublistas.

Conceito

- A solução esboçada anteriormente pode ser expressa pelo algoritmo recursivo

$$\text{smt}(\,L\,) := \ \textbf{se}\ \text{curta}(\,L\,) \quad \textbf{então}\ \text{imd}(\,L\,) \quad \textbf{senão}\ \text{soma}(\,\text{smt}(\,\text{sublist}_1(\,L\,))\ \text{smt}(\,\text{sublist}_2(\,L\,)\,)\,).$$

- Esse algoritmo usa alguns procedimentos como auxiliares:
  - O teste unário *curta* (para ver se L tem comprimento no máximo 1);
  - a função unária *imd* (dando 0 ou o único elemento da lista);
  - as funções unárias $sublist_1$ e $sublist_2$ (partindo L em sublistas);
  - a função binária *soma* (somando resultados: $\text{soma}(r_1, r_2) := r_1 + r_2$.

Conceito

Exemplo: Número de ocorrências de um elemento em uma lista.

- O problema de contar quantas vezes um objeto *obj* ocorre em uma lista L de objetos poderia ser resolvido por divisão e conquista da seguinte maneira:
  - Se a lista L é curta, basta saber se *obj* ocorre em L (dando saída 1) ou não (dando saída 0).
  - Caso contrário, partimos L em sublistas L_1 e L_2 e aplicamos recursivamente o mesmo processo até que tenhamos só sublistas curtas, as quais já sabemos tratar;
  - então bastará combinar as saídas correspondentes a essas sublistas.

Conceito

Exemplo: Classificação de listas por intercalação de sublistas

- Desejamos um algoritmo que receba como entrada uma lista e devolva essa lista classificada.
- Vamos aplicar a ideia de divisão e conquista:
  - Se a lista de entrada é curta, já está classificada, caso contrário a entrada será dividida em duas listas: a primeira metade Prim(d) e a segunda metade Fin(d)
  - Essas duas sublistas serão recursivamente classificadas e intercaladas para assim se obter uma versão classificada da lista de entrada.

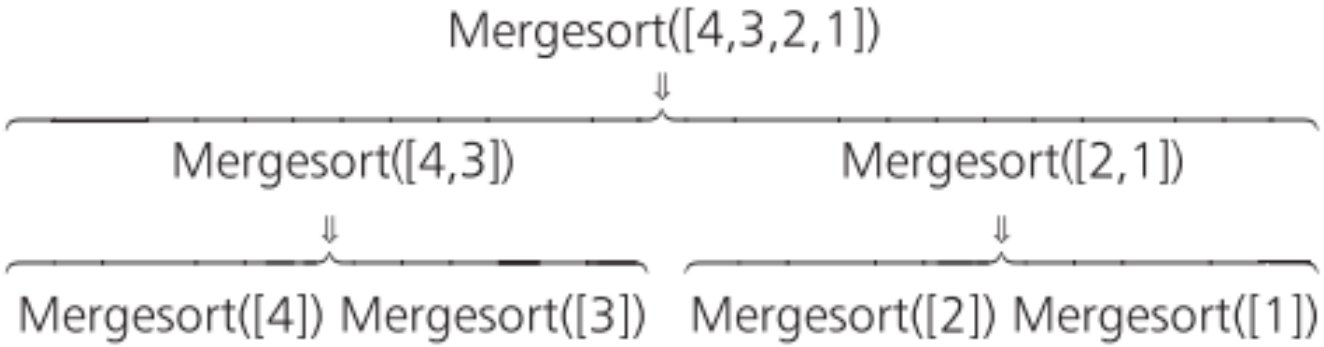
Conceito

Mergesort

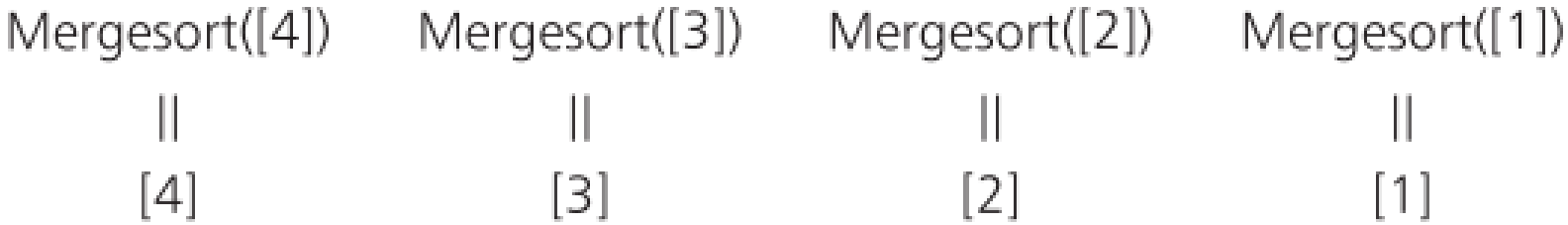
```
0.  n ← compr( d ) ;                                        {comprimento da entrada}
1.  se n ≤ 1                                                {testa se a entrada é curta ou não: 12}
2.     então                                                {n ≤ 1 (saída direta): 5}
3.        r ← d ;                                           {entrada curta já está classificada}
4.        retorne-saída( r )                                {dá saída direta d}
5.     fim-então                                            {2: entrada curta (saída direta)}
6.     senão                                                {n > 1 (recursão): 11}
7.        d1 ← Prim(d) ; d2 ← Fin(d) ;                      {parte em duas metades}
8.        r1 ← Mergesort( d1 ) ; r2 ← Mergesort( d2 ) ;     {recursão}
9.        r ← Intercl( r1 , r2 ) ;                          {intercala partes classificadas}
10.       retorne-saída( r )                                {dá saída intercalada r}
11.    fim-senão                                            {6: entrada não curta (recursão)}
12. fim-se                                                  {1: n ≤ 1?}
13. fim-Função                {fim do algoritmo Mergesort: classificação por intercalação}
```

Conceito

- A execução desse algoritmo Mergesort envolve chamadas recursivas sobre sublistas. Por exemplo, sobre a entrada d := [4, 3, 2, 1], temos



- As folhas dessa árvore têm chamadas sobre sublistas curtas, que já estão classificadas. Assim, obtemos sublistas classificadas diretamente



Conceito

- Finalmente, essas sublistas classificadas são intercaladas, conforme a seguinte árvore:

$$\underbrace{\underbrace{[4]\,[3]}_{\Downarrow\ [3,4]}\quad\underbrace{[2]\,[1]}_{\Downarrow\ [1,2]}}_{[1,2,3,4]}$$

- De maneira geral, a execução do algoritmo Mergesort sobre uma entrada d envolve chamadas recursivas, decompondo a entrada até obter sublistas curtas e intercalando sublistas já classificadas.

Estratégia

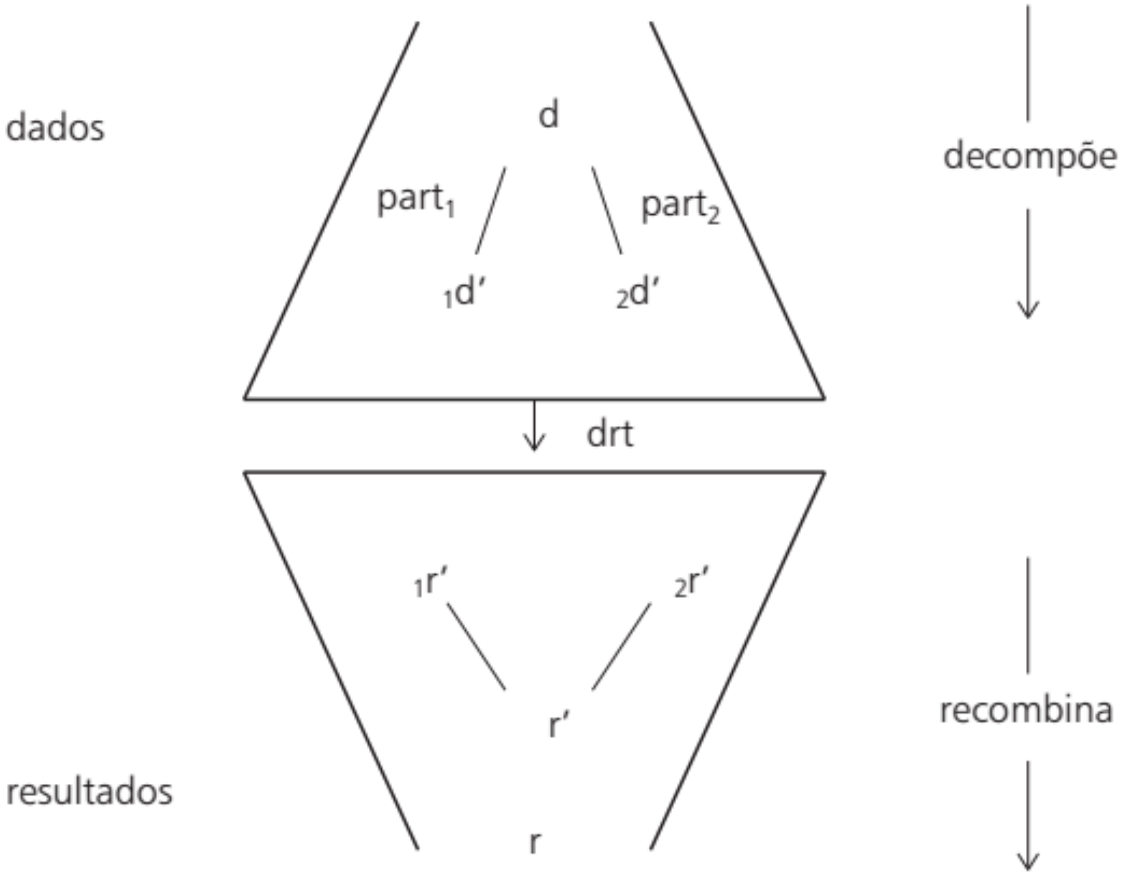
- Decompor o problema em subproblemas independentes, resolvê-los e então combinar as soluções obtidas em uma solução para o problema original.
- Isso dá um processo recursivo de decomposições e recombinações:
  - Dada uma entrada, se ela é suficientemente simples, obtemos diretamente uma saída correspondente.
  - Caso contrário, ela é decomposta em entradas mais simples, para as quais aplicamos o mesmo processo, obtendo saídas correspondentes, que são então combinadas em uma saída para a entrada original.
  - Para isso, usamos procedimentos (funções) como se segue:
    - duas funções unárias $part_1$ e $part_2$ para decompor entradas; e
    - uma função binária cmbn_2 para combinar saídas.

Estratégia

- O processo de decomposição não é necessário para entradas suficientemente simples, para as quais já sabemos como obter saídas correspondentes.
- Podemos modelar tal situação por meio de:
  - um procedimento unário smpl para testar se a entrada é simples ou não; e
  - uma função unária drt dando saídas para entradas simples.

Note...

- Algoritmos de divisão e conquista em geral são recursivos.
- A recursão desempenha importante papel na divisão e conquista.
- O funcionamento da recursão na divisão e conquista pode ser visualizado considerando-se as árvores que o procedimento abstrato Div_Conq_2 (divisão e conquista binária) constrói, em três estágios como se segue



Expandindo

- Imaginamos inicialmente que cada entrada seria decomposta em duas partes.
- Podemos analogamente considerar o caso em que cada entrada será decomposta em um certo número *m* de partes.
- Nesse caso geral, a divisão e conquista usará as seguintes operações e predicados:
  - um procedimento unário smpl para testar se a entrada é simples ou não;
  - uma função unária drt dando saídas para entradas simples;
  - *m* funções unárias part_i, $i = 1, \ldots, m$, para decompor entradas;
  - uma função *m*-ária cmbn_m para combinar *m* saídas.

Exemplos

- Um exemplo imediato de divisão e conquista ternária ($m = 3$) pode ser obtido, modificando-se o algoritmo Quicksort.
- Essa nova versão dividiria a entrada em três por comparação com o pivô:
  - a primeira com os elementos menores do que o pivô,
  - a segunda com os elementos iguais ao pivô,
  - e a terceira com os elementos maiores do que o pivô, em que a segunda parte não requer classificação.
  - Sua combinação seria a concatenação.

Exemplos

- Ainda outro exemplo de divisão e conquista não binária é o **algoritmo de Strassen** para multiplicação de matrizes quadradas.
- Para multiplicar duas matrizes $n \times n$, esse algoritmo decompõe cada uma delas em 4 blocos $(n/2) \times (n/2)$ e calcula 7 produtos de matrizes, dos quais os 4 blocos da matriz produto podem ser obtidos.

Algoritmo de Strassen

- Particiona-se as matrizes em blocos de mesmo tamanho:

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}_{1,1} & \mathbf{A}_{1,2} \\ \mathbf{A}_{2,1} & \mathbf{A}_{2,2} \end{bmatrix}, \mathbf{B} = \begin{bmatrix} \mathbf{B}_{1,1} & \mathbf{B}_{1,2} \\ \mathbf{B}_{2,1} & \mathbf{B}_{2,2} \end{bmatrix}, \mathbf{C} = \begin{bmatrix} \mathbf{C}_{1,1} & \mathbf{C}_{1,2} \\ \mathbf{C}_{2,1} & \mathbf{C}_{2,2} \end{bmatrix}$$

com

$$\mathbf{A}_{i,j}, \mathbf{B}_{i,j}, \mathbf{C}_{i,j} \in R^{2^{n-1} \times 2^{n-1}}.$$

Então

$$\begin{aligned}
\mathbf{C}_{1,1} &= \mathbf{A}_{1,1}\mathbf{B}_{1,1} + \mathbf{A}_{1,2}\mathbf{B}_{2,1} \\
\mathbf{C}_{1,2} &= \mathbf{A}_{1,1}\mathbf{B}_{1,2} + \mathbf{A}_{1,2}\mathbf{B}_{2,2} \\
\mathbf{C}_{2,1} &= \mathbf{A}_{2,1}\mathbf{B}_{1,1} + \mathbf{A}_{2,2}\mathbf{B}_{2,1} \\
\mathbf{C}_{2,2} &= \mathbf{A}_{2,1}\mathbf{B}_{1,2} + \mathbf{A}_{2,2}\mathbf{B}_{2,2}
\end{aligned}$$

Algoritmo de Strassen

- Define-se então um conjunto de 7 matrizes que serão usadas para construir a matriz produto:

$$
\left.
\begin{aligned}
\mathbf{M}_1 &:= (\mathbf{A}_{1,1} + \mathbf{A}_{2,2})(\mathbf{B}_{1,1} + \mathbf{B}_{2,2}) \\
\mathbf{M}_2 &:= (\mathbf{A}_{2,1} + \mathbf{A}_{2,2})\mathbf{B}_{1,1} \\
\mathbf{M}_3 &:= \mathbf{A}_{1,1}(\mathbf{B}_{1,2} - \mathbf{B}_{2,2}) \\
\mathbf{M}_4 &:= \mathbf{A}_{2,2}(\mathbf{B}_{2,1} - \mathbf{B}_{1,1}) \\
\mathbf{M}_5 &:= (\mathbf{A}_{1,1} + \mathbf{A}_{1,2})\mathbf{B}_{2,2} \\
\mathbf{M}_6 &:= (\mathbf{A}_{2,1} - \mathbf{A}_{1,1})(\mathbf{B}_{1,1} + \mathbf{B}_{1,2}) \\
\mathbf{M}_7 &:= (\mathbf{A}_{1,2} - \mathbf{A}_{2,2})(\mathbf{B}_{2,1} + \mathbf{B}_{2,2})
\end{aligned}
\right\}
\Longrightarrow
\left\{
\begin{aligned}
\mathbf{C}_{1,1} &= \mathbf{M}_1 + \mathbf{M}_4 - \mathbf{M}_5 + \mathbf{M}_7 \\
\mathbf{C}_{1,2} &= \mathbf{M}_3 + \mathbf{M}_5 \\
\mathbf{C}_{2,1} &= \mathbf{M}_2 + \mathbf{M}_4 \\
\mathbf{C}_{2,2} &= \mathbf{M}_1 - \mathbf{M}_2 + \mathbf{M}_3 + \mathbf{M}_6
\end{aligned}
\right.
$$

Divisão e Conquista Unária

- No caso de divisão e conquista unária ($m = 1$), o termo '*divisão*' não parece apropriado, '*redução*' sendo talvez mais adequado.
- Algoritmos de busca podem servir para ilustrar esse caso: a busca em uma tabela é reduzida a buscas em subtabelas menores.
  - No caso de busca sequencial, a busca original é reduzida a uma busca numa tabela com um elemento a menos, caso o elemento procurado ainda não tenha sido encontrado.
  - No caso de busca binária em tabela ordenada, a busca original é reduzida a buscar em uma metade da tabela, dependendo da comparação com o elemento procurado.

Projeto de Algoritmos de Divisão e Conquista

- A fim de aplicar a estratégia de divisão e conquista, devemos formular os procedimentos auxiliares de maneira apropriada.
- A divisão da entrada em partes (aproximadamente) iguais costuma levar a árvores menores.
- O chamado *princípio de equipartição* aconselha partir a entrada em partes (aproximadamente) iguais a fim de diminuir a profundidade das árvores, assim *tendendo a melhorar o desempenho do algoritmo resultante*.
- Vamos a um exemplo: Máximo e mínimo de tabela (divisão e conquista binária)

Exemplo

```
0.  n ← q − p + 1 ;                                   {tamanho da tabela}
1.  se n ≤ 2                                          {testa se a tabela é pequena ou não: 23}
2.      então                                         {n ≤ 2 (saída direta): 14}
3.          se n = 1                                  {n = 1?: 12}
4.              então                                 {n=1:7}
5.                  r_M ← Tab[ p ] ;                  {máximo}
6.                  r_m ← Tab[ p ]                    {mínimo}
7.              fim-então                             {4: n = 1}
8.              senão                                 {n = 2:11}
9.                  r_M ← Máx( Tab[ p ] , Tab[ q ] ) ;  {máximo}
10.                 r_m ← mín( Tab[ p ] , Tab[ q ] ) ;  {mínimo}
11.             fim-senão                             {8: n = 2}
12.         fim-se ;                                  {3: n = 1?}
13.         retorne-saída( r_M , r_m )                {dá saída direta}
14.     fim-então                                     {2: tabela pequena (saída direta)}
15.     senão                                         {n > 2 (recursão): 22}
16.         k ← ⌈ n / 2 ⌉ ; q_i ← p + k ; p_s ← q_i + 1 ;  {duas metades}
17.         ( s_M , s_m ) ← Max_min( Tab[ p .. p_s ] ) ;  {recursão: 1ª metade}
18.         ( t_M , t_m ) ← Max_min( Tab[ q_i .. q ] ) ;  {recursão: 2ª metade}
19.         r_M ← Máx( s_M , t_M ) ;                  {combinação: máximo}
20.         r_m ← mín( s_m , t_m ) ;                  {combinação: mínimo}
21.         retorne-saída( r_M , r_m )                {dá saída combinada}
22.     fim-senão                                     {15: tabela não pequena (recursão)}
23. fim-se                                            1: n ≤ 2?}
24. fim-Função                                        {fim do algoritmo Max_min: máximo e mínimo de tabela}
```

Execute manualmente para:

(i) [1, 4, 5, 2, 3, 5]
(ii) [10, 4, 3, 5, 20, 30, 50, 2, 6, 7].

Projeto de Algoritmos de Divisão e Conquista

- Vamos a um exemplo de divisão e conquista unária
- Consideremos uma versão simplificada do problema de busca em tabela: determinar se uma chave ocorre ou não em uma tabela.
- Considere o algoritmo Oc_seq a seguir.
- Execute manualmente, construindo as árvores de execução, para determinar se a chave 20 ocorre em cada uma das seguintes tabelas:

  (i) [10, 4, 3, 5, 20, 30, 50, 2, 6, 7];

  (ii) [10, 41, 25, 2, 3, 5].

Exemplo

```
0.  n ← q − p + 1 ;                                   {tamanho da tabela}
1.  se n = 1                                          {testa se a tabela é pequena ou não: 17}
2.      então                                         {n = 1 (saída direta): 8}
3.          se tab[ p ] = ch                          {tab[ p ] = ch?: 6}
4.              então achou ← V                       {tab[ p ] = ch}
5.              senão  achou ← F                      {tab[ p ] ≠ ch}
6.          fim-se ;                                  {3: tab[ p ] = ch?}
7.          retorne-saída( achou )                    {dá saída direta}
8.      fim-então                                     {2: tabela pequena (saída direta)}
9.      senão                                         {n > 1 (recursão): 16}
10.         se tab[ p ] = ch                          {tab[ p ] = ch?: 13}
11.             então q ← p                           {tab[ p ] = ch}
12.             senão p ← p + 1                       {tab[ p ] ≠ ch}
13.         fim-se                                    {10: tab[ p ] = ch?}
14.         achou ← Oc_seq( ch : Elm , tab : Cnj ) ;  {recursão: parte}
15.         retorne-saída( achou )                    {retorna saída}
16.     fim-senão                                     {9: tabela não pequena (recursão)}
17. fim-se                                            {1: n = 1?}
18. fim-Função          {fim do algoritmo Oc_seq: ocorrência em tabela (sequencial)}
```

Complexidade de Algoritmos de Divisão e Conquista

- Consideremos o algoritmo Oc_seq (para determinar se uma chave ocorre numa tabela por busca sequencial).
- Vimos que esse algoritmo foi obtido como instância da formulação Div_Conq_1 (divisão e conquista unária).
- Examinando o decréscimo de tamanho a cada chamada recursiva, podemos estimar a ordem da complexidade pessimista como $c_P[\ Oc_seq\ ](n) = O(n)$.

Complexidade de Algoritmos de Divisão e Conquista

- Consideremos o algoritmo Mergesort (para classificar listas por intercalação de sublistas).
- Vimos que esse algoritmo é uma instância da formulação Div_Conq_2 da divisão e conquista binária.
- Examinando o decréscimo de tamanho nas chamadas recursivas, podemos estimar a complexidade pessimista como $c_P[\text{ Mergesort }](n) = O(n \log_2 n)$.
  - ***Note***: só a linha 9 envolve comparações: $O(n)$. Cada invocação recursiva divide a lista aproximadamente ao meio, assim serão feitas $O(\log_2 n)$ chamadas para uma lista de tamanho n.}

Observações

- Frequentemente, "simples" significa "ter tamanho pequeno".
- No exemplo de intercalação, uma lista simples é uma lista curta (com tamanho máximo 1). Isso nos fornece um *limiar* separando os dados simples dos não simples: um limiar é um natural ***a*** tal que smpl(d) se e somente se tam(d) ≤ a.
- O algoritmo mergesort tem limiar a = 1.
- É bastante comum o caso em que a recombinação de resultados não diminui tamanho.
- Um algoritmo de divisão e conquistas será dito *discriminante* se somente se ele satisfaz a propriedade de não diminuir de tamanho e ter um limiar.
- Finalmente outro aspecto importante é o esforço de recursão: teste de simplicidade, decomposição e recombinação. Um algoritmo de divisão e conquista terá grau *k* quando seu esforço de recursão tiver ordem $O(n^k)$.

Observações

Dois casos típicos de decréscimo de tamanho são:

- por subtração de $\varepsilon > 0$: $S(d) = tam(d) - \varepsilon$
- por divisão por $c > 1$: $S(d) = \lceil tam(d) / c \rceil$.

A altura das árvores (profundidade de recursão) vem a ser respectivamente:
$H(d) = \lceil (tam(d) - a) / \varepsilon \rceil$ e $H(d) = \lceil \log_c(tam(d) - a) \rceil$.

- Obs: *a* é o limiar.

Dúvidas?